`define_static_{string,object,array}`

Document #:	P3491R0 [Latest] [Status]
Date:	2024-12-15
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Wyatt Childers <wcc@edg.com> Peter Dimov <pdimov@gmail.com> Dan Katz <dkatz85@bloomberg.net> Barry Revzin <barry.revzin@gmail.com> Daveed Vandevoorde <daveed@edg.com>

Contents

1 Introduction
2 Proposal
- 2.1 The Overlapping Question
- 2.2 The Structural Question
- 2.3 Possible Implementation
- 2.4 Examples
  - 2.4.1 Use as non-type template parameter
  - 2.4.2 Pretty-printing
  - 2.4.3 Promoting Containers
  - 2.4.4 With Expansion Statements
  - 2.4.5 Implementing `source_location`
- 2.5 Related Papers in the Space

3 Wording
- 3.1 Feature-Test Macro

4 References

§ 1 Introduction

These functions were originally proposed as part of [P2996R7], but are being split off into their own paper.

There are situations where it is useful to take a string (or array) from compile time and promote it to static storage for use at runtime. We currently have neither:

— non-transient constexpr allocation (see [P1974R0], [P2670R1]), nor

— generalized support for class types as non-type template parameters (see [P2484R0], [P3380R1])

If we had non-transient constexpr allocation, we could just directly declare a static constexpr variable. And if we could use these container types like `std::string` and `std::vector<T>` as non-type template parameter types, then we would use those directly too.

But until we have such a language solution, people have over time come up with their own workarounds. For instance, Jason Turner in a recent talk presents what he calls the "constexpr two-step." It's a useful pattern, although limited and cumbersome (it also requires specifying a maximum capacity).

Similarly, the lack of general support for non-type template parameters means we couldn't have a `std::string` template parameter (even if we had non-transient constexpr allocation), but promoting the contents of a string to an external linkage, static storage duration array of `const char` means that you can use a pointer to that array as a non-type template parameter just fine.

So having facilities to solve these problems until the general language solution arises is very valuable.

§ 2 Proposal

This paper proposes three new additions — `std::define_static_string`, `std::define_static_object`, and `std::define_static_array`, as well as a helper function for dealing with string literals:

```
namespace std {
  consteval auto is_string_literal(char const* p) -> bool;
  consteval auto is_string_literal(char8_t const* p) -> bool;

  template <ranges::input_range R> // only if the value_type is char or
        char8_t
    consteval auto define_static_string(R&& r) -> ranges::range_val-
        ue_t<R> const*;

  template <class T>
   consteval auto define_static_object(T&& v) -> remove_reference_t<T>
        const*;

  template <ranges::input_range R>
   consteval auto define_static_array(R&& r) -> span<ranges::range_val-
        ue_t<R> const>;
}
```

`is_string_literal` takes a pointer to either `char const` or `char8_t const`. If it's a pointer to either a string literal `V` or a subobject thereof, these functions return `true`. Otherwise, they return `false`. Note that we can't necessarily return a pointer to the start of the string literal because in the case of overlapping string literals — how do you know which pointer to return?

`define_static_string` is limited to ranges over `char` or `char8_t` and returns a `char const*` or `char8_t const*`, respectively. They return a pointer instead of a `string_view` (or `u8string_view`)

specifically to make it clear that they return something null terminated. If `define_static_string` is passed a string literal that is already null-terminated, it will not be doubly null terminated.

`define_static_array` exists to handle the general case for other types, and now has to return a `span` so the caller would have any idea how long the result is. This function requires that the underlying type `T` be structural.

`define_static_object` is a special case of `define_static_array` for handling a single object. Technically, `define_static_object(v)` can also be achieved via `define_static_array(views::single(v)).data()`, but it's has its own use as we'll show.

Technically, `define_static_array` can be used to implement `define_static_string`:

```
consteval auto define_static_string(string_view str) -> char const* {
        return    define_static_array(views::concat(str,    views::sin-
        gle('\0'))).data();
}
```

But that's a fairly awkward implementation, and the string use-case is sufficiently common as to merit a more ergonomic solution.

There are two design questions that we have to address: whether objects can overlap and whether `define_static_array` needs to mandate structural.

§ 2.1 The Overlapping Question

Consider the existence of `template <char const*> struct C;` and the following two translation units:

TU #1	TU #2
`C<define_static_string("dedup")> c1;` `C<define_static_string("dup")> c2;`	`C<define_static_string("holdup")> c3;` `C<define_static_string("dup")> c4;`

In the specification in [P2996R7], the results of `define_static_string` were allowed to overlap. That is, a possible result of this program could be:

TU #1	TU #2

`inline char const __arr_dedup[] =`
`"dedup";`
`C<__arr_dedup> c1;`
`C<__arr_dedup + 2> c2;`
	`inline char const __arr_holdup[] =`
	`"holdup";`
	`C<__arr_holdup> c3;`
	`C<__arr_holdup + 3> c4;`

This means whether `c2` and `c4` have the same type is unspecified. They could have the same type if the implementation chooses to not overlap (or no overlap is possible). Or they could have different types.

However, that's not the right way to think about overlapping.

A more accurate way to present the ability to support overlapping arrays from `define_static_string` would be that the two TUs would merge more like this:

<table>
<tr><th>TU #1</th><th>TU #2</th></tr>
<tr><td colspan="2">

```
// all our static strings merged
inline constexpr char const __arr_dedup[] = "dedup";
inline constexpr char const __arr_holdup[] = "holdup";

// this behaves like an array for all purposes, including that
// __arr_dup[-1] is not a valid constant expression (because out of bounds)
// but the implementation is allowed to have &__arr_dup[0] == &__arr_dedup[2]
inline constexpr char const __arr_dup[] = "dup";
```

</td></tr>
<tr><td>

```
// C<define_static_string("dedup")>
C<__arr_dedup> c1;

// C<define_static_string("dup")>
C<__arr_dup> c2;
```

</td><td>

```
// C<define_static_string("holdup")>
C<__arr_holdup> c3;

// C<define_static_string("dup")>
C<__arr_dup> c4;
```

</td></tr>
</table>

At this point, the usual template-argument-equivalence rules apply, so `c4` and `c2` would definitely have the same type, because their template arguments point to the same array. As desired.

The one thing we really have to ensure with this route, as pointed out by Tomasz Kamiński, is that comparison between distinct non-unique objects needs to be unspecified. This is so that you cannot ensure overlap. In other words:

```
constexpr char const* a = define_static_string("dedup");
constexpr char const* b = define_static_string("dup");

static_assert(b == b);                                  // ok, #1
static_assert(b + 1 == b + 1);                          // ok, #2
static_assert(a != b);                                  // ok, #3
static_assert(a + 2 != b);                                      // error:
        unspecified
static_assert(string_view(a + 2) == string_view(b)); // ok, #4
```

Now, it had better be the case that `b == b` and `b + 1 == b + 1` are both valid checks. It would be fairly strange otherwise. Similarly, the goal is to not be able to observe whether `a + 2 == b`. It could be `true` at runtime. Or not. We have no idea at compile-time yet, so it'd be better to just not even allow an answer.

The interesting one is `a != b`. We could say that the comparison is unspecified (because they're pointers into distinct non-unique objects). But in this case, regardless of whether `a` and `b` overlap, `a != b` is *definitely* going to be `true` at runtime. So we should only make unspecified the case that we actually cannot specify. After all, it would be strange if `a == b` were unspecified but `a[0] == b[0]` was `false`.

Note that, regardless, the `string_view` comparison is valid, since that is comparing the contents.

This does present an interesting situation where `a == b` could be invalid but `is_same_v<C<a>, C<b>>` would be valid.

§ 2.2 The Structural Question

For `define_static_string`, we have it easy because we know that `char` and `char8_t` are both structural types. But for `define_static_array`, we get an arbitrary `T`. How can we produce overlapping arrays in this case? If `T` is structural, we can easily ensure that equal invocations produce the *same* `span` result.

But if `T` is not structural, we have a problem, because `T*` is, regardless. So we have to answer the question of what to do with:

```
template <auto V> struct C { };

C<define_static_array(r).data()> c1;
C<define_static_array(r).data()> c2;
```

Either:

— this works, and it is unspecified whether `c1` and `c2` have the same type.

— the call to `define_static_array` works, but the resulting pointer is not usable as a non-type template argument (in the same way that string literals are not).

— the call to `define_static_array` mandates that the underlying type is structural.

None of these options is particularly appealing. The last prevents some very motivating use-cases since neither `span` nor `string_view` are structural types yet, which means you cannot reify a `vector<string>` into a `span<string_view>`, but hopefully that can be resolved soon ([P3380R1]). You can at least reify it into a `span<char const*>`?

For now, this paper proposes the last option, as it's the simplest (and the relative cost will hopefully decrease over time). Allowing the call but rejecting use as non-type template parameters is appealing though.

§ 2.3 Possible Implementation

`define_static_string` can be nearly implemented with the facilities in [P2996R7], we just need `is_string_literal` to handle the different signature proposed in this paper.

`define_static_array` for is similar:

```
template <auto V>
inline constexpr auto __array = V.data();

template <size_t N, class T, class R>
consteval auto define_static_string_impl(R& r) -> T const* {
    array<T, N+1> arr;
```

```
    ranges::copy(r, arr.data());
    arr[N] = '\0'; // null terminator
     return extract<T const*>(substitute(^^__array, {meta::reflect_val-
        ue(arr)}));
}

template <ranges::input_range R>
consteval auto define_static_string(R&& r) -> ranges::range_value_t<R>
        const* {
    using T = ranges::range_value_t<R>;
    static_assert(std::same_as<T, char> or std::same_as<T, char8_t>);

    if constexpr (not ranges::forward_range<R>) {
        return define_static_string(ranges::to<std::vector>(r));
    } else {
        if constexpr (requires { is_string_literal(r); }) {
            // if it's an array, check if it's a string literal and ad-
        just accordingly
            if (is_string_literal(r)) {
                return define_static_string(basic_string_view(r));
            }
        }

        auto impl = extract<auto(*)(R&) -> T const*>(
            substitute(^^define_static_string_impl,
                       {
                           meta::reflect_value(ranges::distance(r)),
                           ^^T,
                           remove_reference(^^R)
                       }));
        return impl(r);
    }
}
```

Demo.

Note that this implementation gives the guarantee we talked about in the previous section. Two invocations of `define_static_string` with the same contents will both end up returning a pointer into the same specialization of the (extern linkage) variable template `__array<V>`. We rely on the mangling of `V` (and `std::array` is a structural type if `T` is, which `char` and `char8_t` are) to ensure this for us. This won't ever produce overlapping arrays, would need implementation help for that, but it is a viable solution for all use-cases.

§ 2.4 Examples

§ 2.4.1 Use as non-type template parameter

```
template <const char *P> struct C { };

const char msg[] = "strongly in favor";  // just an idea..

C<msg> c1;                              // ok
```

```
C<"nope"> c2;                          // ill-formed
C<define_static_string("yay")> c3;  // ok
```

§ 2.4.2 Pretty-printing

In the absence of general support for non-transient constexpr allocation, such a facility is essential to building utilities like pretty printers.

An example of such an interface might be built as follow:

```
template <std::meta::info R> requires is_value(R)
  consteval auto render() -> std::string;

template <std::meta::info R> requires is_type(R)
  consteval auto render() -> std::string;

template <std::meta::info R> requires is_variable(R)
  consteval auto render() -> std::string;

// ...

template <std::meta::info R>
consteval auto pretty_print() -> std::string_view {
  return define_static_string(render<R>());
}
```

This strategy lies at the core of how the Clang/P2996 fork builds its example implementation of the `display_string_of` metafunction.

§ 2.4.3 Promoting Containers

In the Jason Turner talk cited earlier, he demonstrates an example of taking a function that produces a `vector<string>` and promoting that into static storage, in a condensed way so that the function

```
constexpr std::vector<std::string> get_strings() {
    return {"Jason", "Was", "Here"};
}
```

Gets turned into an array of string views. We could do that fairly straightforwardly, without even needing to take the function `get_strings()` as a template parameter:

```
consteval auto promote_strings(std::vector<std::string> vs)
    -> std::span<std::string_view const>
{
    // promote the concatenated strings to static storage
    std::string_view promoted = std::define_static_string(
        std::ranges::fold_left(vs, std::string(), std::plus()));
```

```
    // now build up all our string views into promoted
    std::vector<std::string_view> views;
    for (size_t offset = 0; std::string const& s : vs) {
        views.push_back(promoted.substr(offset, s.size()));
        offset += s.size();
    }

    // promote our array of string_views
    return std::define_static_array(views);
}

constexpr auto views = promote_strings(get_strings());
```

Or at least, this will work once `string_view` becomes structural. Until then, this can be worked around with a `structural_string_view` type that just has public members for the data and length with an implicit conversion to `string_view`.

§ 2.4.4 With Expansion Statements

Something like this ([P1306R2]) is not doable without non-transient constexpr allocation :

```
constexpr auto f() -> std::vector<int> { return {1, 2, 3}; }

consteval void g() {
    template for (constexpr int I : f()) {
        // doesn't work
    }
}
```

But if we promote the contents of `f()` first, then this would work fine:

```
consteval void g() {
    template for (constexpr int I : define_static_array(f())) {
        // ok!
    }
}
```

§ 2.4.5 Implementing `source_location`

One interesting use of a specific `define_static_object` (for the single object case), courtesy of Richard Smith, is to implement the single-pointer optimization for `std::source_location` without compiler support:

```
class source_location {
    struct impl {
        char const* filename;
        int line;
    };
```

```
    impl const* p_;

public:
    static consteval auto current(char const* file = __builtin_FILE(),
                                  int line = __builtin_LINE()) noexcept
        -> source_location
    {
        // first, we canonicalize the file
          impl data = {.filename = define_static_string(file), .line =
        line};

        // then we canonicalize the data
        impl const* p = define_static_object(data);

        // and now we have an external linkage object mangled with this
        location
        return source_location{p};
    }
};
```

§ 2.5 Related Papers in the Space

A number of other papers have been brought up as being related to this problem, so let's just enumerate them.

— [P3094R5] proposed `std::basic_fixed_string<char, N>`. It exists to solve the problem that `C<"hello">` needs support right now. Nothing in this paper would make `C<"hello">` work, although it might affect the way that you would implement the type that makes it work.

— [P3380R1] proposes to extend non-type template parameter support, which could eventually make `std::string` usable as a non-type template parameter. But without non-transient constexpr allocation, this doesn't obviate the need for this paper. Note that that paper depends on this paper for how to normalize string literals, making string literals usable as non-type template arguemnts.

— [P1974R0] and [P2670R1] propose approaches to tackle the non-transient allocation problem.

Given non-transient allocation *and* a `std::string` and `std::vector` that are usable as non-type template parameters, this paper likely becomes unnecessary. Or at least, fairly trivial:

```
template <auto V>
inline constexpr auto __S = V.c_str();

template <ranges::input_range R>
consteval auto define_static_string(R&& r) -> ranges::range_value_t<R>
        const* {
    using T = ranges::range_value_t<R>;
    static_assert(std::same_as<T, char> or std::same_as<T, char8_t>);

    auto S = ranges::to<basic_string<T>>(r);
```

```
        return extract<T const*>(substitute(^^__S, {meta::reflect_val-
        ue(S)}));
}
```

§ 3 Wording

Change 6.7.2 [intro.object]:

9 An object is a *potentially non-unique object* if it is

(9.1) — a string literal object ([lex.string]),

(9.2) — the backing array of an initializer list ([dcl.init.ref]),

(9.3) — the result of a call to `std::define_string` or `std::define_array`, or

(9.4) — a subobject thereof.

Change 7.6.10 [expr.eq]/3:

3 If at least one of the operands is a pointer, pointer conversions, function pointer conversions, and qualification conversions are performed on both operands to bring them to their composite pointer type. Comparing pointers is defined as follows:

(3.1) — If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object, the result of the comparison is unspecified.

(3.1b) — Otherwise, if the pointers point into distinct potentially non-unique objects ([intro.object]) with the same contents, the result of the comparison is unspecified.

[*Example 1:*

```
constexpr char const* a = std::define_static_string("other");
constexpr char const* b = std::define_static_string("another");

static_assert(a != b);     // OK
static_assert(a == b + 2); // error: unspecified
static_assert(b == b);     // OK
```

— *end example*]

(3.2) — Otherwise, if the pointers are both null, both point to the same function, or both represent the same address, they compare equal.

(3.3) — Otherwise, the pointers compare unequal.

Add to [meta.syn]:

```
namespace std {
+ // [meta.string.literal], checking string literals
+ consteval bool is_string_literal(const char* p);
+ consteval bool is_string_literal(const char8_t* p);

+ // [meta.define.static], promoting to runtime storage
+ template <ranges::input_range R>
+    consteval const ranges::range_value_t<R>* define_static_string(R&&
        r);
+
+ template <class T>
+   consteval const remove_reference_t<T>* define_static_object(T&& r);
+
+  template <ranges::input_range R>
+                  consteval    span<const    ranges::range_value_t<R>>
        define_static_array(R&& r);
}
```

Add to the new clause [meta.string.literal]:

```
  consteval bool is_string_literal(const char* p);
  consteval bool is_string_literal(const char8_t* p);
```

1 *Returns*: If `p` points to a string literal or a subobject thereof, `true`. Otherwise, `false`.

Add to the new clause [meta.define.static]

1 The functions in this clause are useful for promoting compile-time storage into runtime storage.

```
  template <ranges::input_range R>
  consteval const ranges::range_value_t<R>* define_static_string(R&& r);
```

2 Let *`CharT`* be `ranges::range_value_t<R>`.

3 *Mandates*: *`CharT`* is either `char` or `char8_t`.

4 Let *`V`*`...` be the pack of elements of type *`CharT`* in `r`. If `r` is a string literal, then *`V`* does not include the trailing null terminator of `r`.

5 Let *`P`* be the template parameter object ([temp.param]) of type `const` *`CharT`*`[sizeof...(V)+1]` initialized with `{V..., `*`CharT`*`()}`.

6 *Returns*: *`P`*.

7 [*Note 1: `P`* is a potentially non-unique object ([intro.object]) — *end note*]

```
  template <class T>
  consteval const remove_reference_t<T>* define_static_object(T&& t);
```

Let `U` be `remove_cvref_t<T>`.

Mandates: `U` is a structural type ([temp.param]) and `constructible_from<U, T>` is `true`.

Let *P* be the template parameter object ([temp.param]) of type `const U` initialized with `t`.

Returns: `std::addressof(P)`.

```
template <ranges::input_range R>
consteval span<const ranges::range_value_t<R>> define_static_array(R&&
        r);
```

Let *T* be `ranges::range_value_t<R>`.

Mandates: *T* is a structural type ([temp.param]) and `constructible_from<T, ranges::range_reference_t<R>>` is `true` and `copy_constructible<T>` is `true`.

Let *V* be the pack of elements of type *T* constructed from the elements of `r`.

Let *P* be the template parameter object ([temp.param]) of type `const T[sizeof...(V)]` initialized with `{V...}`.

Returns: `span<const T>(P)`.

[*Note 2: P* is a potentially non-unique object ([intro.object]) — *end note*]

§ 3.1 Feature-Test Macro

Add to 17.3.2 [version.syn]:

```
#define __cpp_lib_define_static 2024XX // freestanding, also in <meta>
```

§ 4 References

[P1306R2] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2024-05-07. Expansion statements.
https://wg21.link/p1306r2

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.
https://wg21.link/p1974r0

[P2484R0] Richard Smith. 2021-11-17. Extending class types as non-type template parameters.
https://wg21.link/p2484r0

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.
https://wg21.link/p2670r1

[P2996R7] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-13. Reflection for C++26.
https://wg21.link/p2996r7

[P3094R5] Mateusz Pusz. 2024-10-15. std::basic_fixed_string.
https://wg21.link/p3094r5
[P3380R1] Barry Revzin. 2024-12-04. Extending support for class types as non-type template parameters.
https://wg21.link/p3380r1